

```

class Solution { public: bool isValid(string s) { stack st1;

for (int i = 0; i < s.size(); i++) {

if (s[i] == '(' || s[i] == '[' || s[i] == '{') { st1.push(s[i]); }

else{

if (st1.empty()) return false;

if (s[i] == ')' && st1.top() == '(' || s[i] == ']' && st1.top() == '[' || s[i] == '}' && st1.top() == '{') {
st1.pop(); }

else { return false; }

} } return st1.empty(); } };

// ===== Gpt response
=====

```

You're **very close**, but your `continue` is causing the problem.

You wrote:

```

else {
    continue;
}

```

This means:

- If it's an opening bracket → push it.
- If it's a closing bracket → `continue` immediately.

So your matching logic **never runs for closing brackets**.

You need to do the opposite: **only check matching logic for closing brackets**.

Fixed version of your approach

```
class Solution {
public:
    bool isValid(string s) {
        stack<char> st1;

        for (int i = 0; i < s.size(); i++) {

            // Opening bracket
            if (s[i] == '(' || s[i] == '[' || s[i] == '{') {
                st1.push(s[i]);
            }

            // Closing bracket
            else {

                if (st1.empty()) {
                    return false;
                }

                if ((s[i] == ')' && st1.top() == '(') ||
                    (s[i] == ']' && st1.top() == '[') ||
                    (s[i] == '}' && st1.top() == '{')) {

                    st1.pop();
                }
                else {
                    return false;
                }
            }
        }

        return st1.empty();
    }
};
```

The key mistake in your code

Your code:

```
if (opening) {
    push();
}
else {
    continue; // ✗ skips closing brackets
}

if (st1.empty()) ...
```

Because of `continue`, the code never reaches:

```
if (s[i] == ')' && ...)
```

for a closing bracket.

You should think of it as:

If opening bracket

→ PUSH

Otherwise (closing bracket)

→ Check if stack is empty

→ Compare with stack top

→ POP if matching

→ Return false if not matching

Your **stack approach is correct** now; you just needed to put the matching logic inside the

`else` block.